



ALGORYTMY I STRUKTURY DANYCH

Termin lab.: piątek 13:30

Kierunek: Informatyka

Grupa lab.: 5

SPRAWOZDANIE Z LABORATORIUM

2. Złożone struktury danych

Autor:

PRO100KatYT

Prowadzący zajęcia:

mgr inż. Domomod

Rok akademicki 2025/2026

Spis treści

1	O programie	2
1.1	Opis	2
1.2	Prezentacja działania programu	2
2	Wykresy benchmarków	3
3	Podsumowanie	3

Zapraszam do lektury poczynając od strony następnej.

1 O programie

1.1 Opis

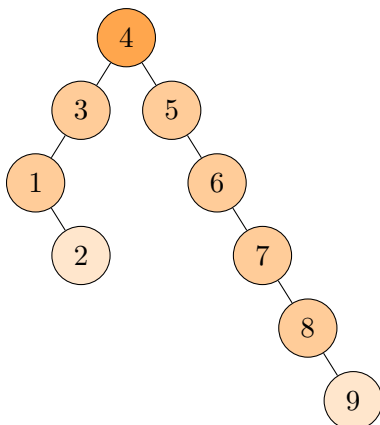
Napisany w Pythonie program służy do generowania i testowania wydajności drzew BST oraz AVL na podstawie danych podanych przez użytkownika. Umożliwia on wykonywanie kilku operacji na tych strukturach, np. wypisywanie węzłów, szukanie min/max czy równoważenie i dokonuje benchmarku wybranych funkcji. Zaimplementowana została także opcja exportu drzewa do TikZ, celem łatwej wizualizacji drzew.

1.2 Prezentacja działania programu

Drzewo BST

```
Terminal

python main.py --tree BST
nodes> 9
insert> 4 5 6 3 7 1 2 8 9
action> Print
Pre-order: 4, 3, 1, 2, 5, 6, 7, 8, 9
In-order: 1, 2, 3, 4, 5, 6, 7, 8, 9
Post-order: 2, 1, 3, 9, 8, 7, 6, 5, 4
action> FindMinMax
Min: 1
Max: 9
action> Remove
remove> 2 1 3 7
action> Rebalance
action> Print
Pre-order: 8, 5, 4, 6, 9
In-order: 4, 5, 6, 8, 9
Post-order: 4, 6, 5, 9, 8
action> Delete
action> Print
You deleted the tree. You cannot Print!
Why are we still here? Just to suffer?
action> Exit
Program exited with status: 0
```

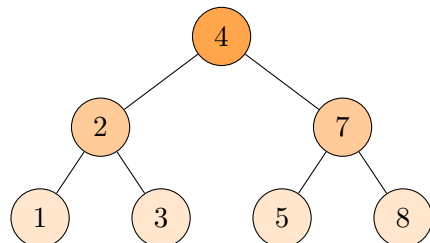


Rysunek 1: Drzewo BST Boże, przed balansem, całe te.

Drzewo AVL

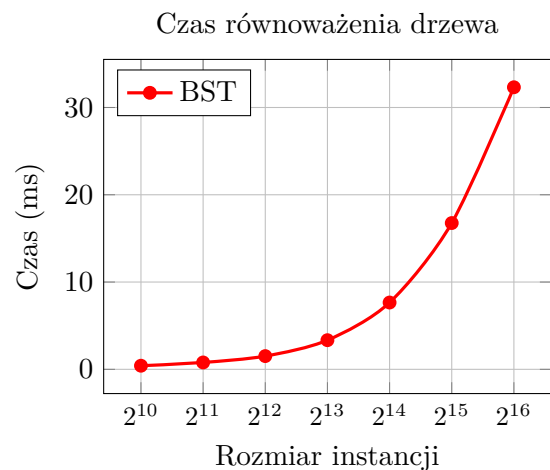
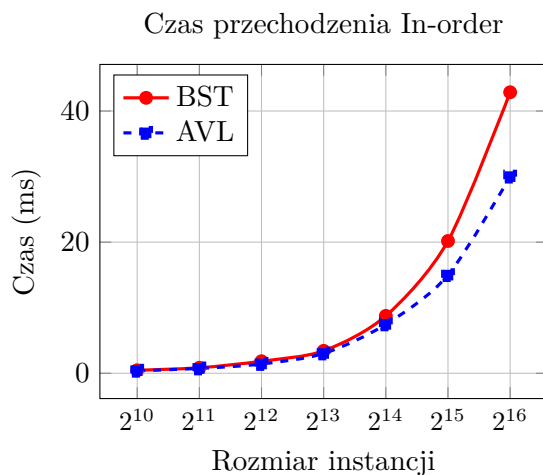
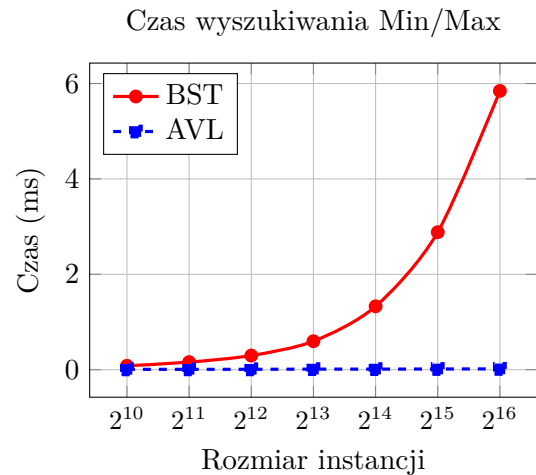
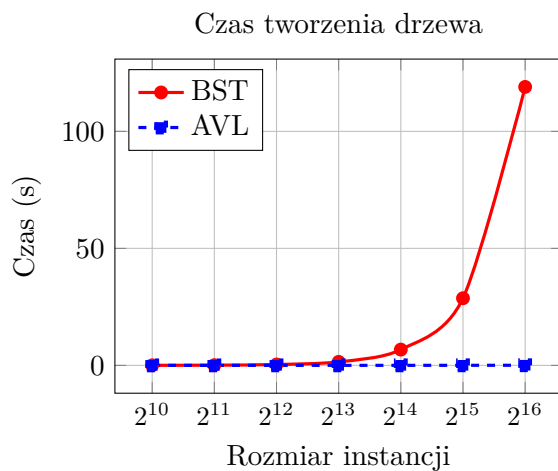
```
Terminal

python main.py --tree AVL
nodes> 9
insert> 1 2 3 4 5 6 7 8 9
action> Print
Pre-order: 5, 2, 1, 3, 4, 7, 6, 8, 9
In-order: 1, 2, 3, 4, 5, 6, 7, 8, 9
Post-order: 1, 4, 3, 2, 6, 9, 8, 7, 5
action> FindMinMax
Min: 1
Max: 9
action> Remove
remove> 6 9
action> Rebalance
action> Print
Pre-order: 4, 2, 1, 3, 7, 5, 8
In-order: 1, 2, 3, 4, 5, 7, 8
Post-order: 1, 3, 2, 5, 8, 7, 4
action> Export
Exported tree:
\node {4} child { node {2} child { node {1} } child { node {3} } } child { node {7} child { node {5} } } child { node {8} } };
action> Delete
action> Print
You deleted the tree. You cannot Print!
Why are we still here? Just to suffer?
action> Exit
Program exited with status: 0
```



Rysunek 2: Drzewo AVL, ogółem.

2 Wykresy benchmarków



3 Podsumowanie

W sprawozdaniu przeanalizowałem dwie struktury danych: binarne drzewo poszukiwań oraz drzewo AVL. Porównałem ich wydajność podczas tworzenia, wyszukiwania minimum i maximum oraz przechodzenia in-order. Wyniki benchmarków jednoznacznie pokazały przewagę drzewa AVL, które dzięki automatycznemu równoważeniu zachowuje wysoką szybkość działań nawet przy dużej liczbie elementów. Potwierdziłem również, że dla danych posortowanych zwykłe drzewo BST staje się nieefektywne ponieważ degeneruje się do postaci winorośli. Zadanie pozwoliło w praktyce zobaczyć różnice w złożoności obu struktur, zrozumieć m.in. działanie rotacji w drzewach zbalansowanych oraz podszkolić umiejętności programistyczne.